Escuela Politécnica Nacional

Facultad de Ingeniería en Sistemas.

Integrantes:

Betancourt Enrique;

Navarrete Alejandro &

Salazar Ambar.

Mini-informe ejercicio 02

Compiladores y Lenguajes, IDSD413

GR01CD

Ing. Rodrigo Chancusig MSc.

05 de noviembre del 2026

El ejercicio construye un programa que siempre que se encuentre la palabra 'pare' en la entrada, se ejecuta la acción establecida en la llamada a printf (), en este caso: 'comando pare recibido'. Lo mismo cuando encuentre la palabra 'inicio', se ejecuta la acción de 'comando inicio ejecutado'. Todo esto deberá ejecutarse analizando toda la entrada de texto en el programa o recorriendo todo el archivo de prueba.

1. Objetivo y funcionamiento

El objetivo es reconocer dos tipos de entrada: la palabra 'pare' e 'inicio', una vez que estas dos entradas sean reconocidas se deberá mostrar ciertas salidas como forma de verificación, en este caso son: 'comando pare recibido' y 'comando inicio ejecutado'.

El funcionamiento está dado por las reglas:

```
%%
pare printf("Comando␣pare␣recibido\n");
inicio printf("Comando␣inicio␣ejecutado\n");
%%
```

Listing 1: Reglas de funcionamiento.

Las cuales son las que detectan la aparición o la ocurrencia de las entradas 'pare' e 'inicio'. De ser detectadas, el programa las remplaza por el contenido de aquellos **printf** . Esto se desarrolla por toda la entrada de texto que se inserte de inicio, o de ser el caso recorriendo todo el archivo de texto con el que se elija ejecutar el programa.

2. Preparación y compilación

Primero se crea la carpeta de trabajo y se guarda la especificación en `dos.l`. El comando `cat dos.l` permite revisar el archivo y comprobar que contiene las reglas y la función principal.

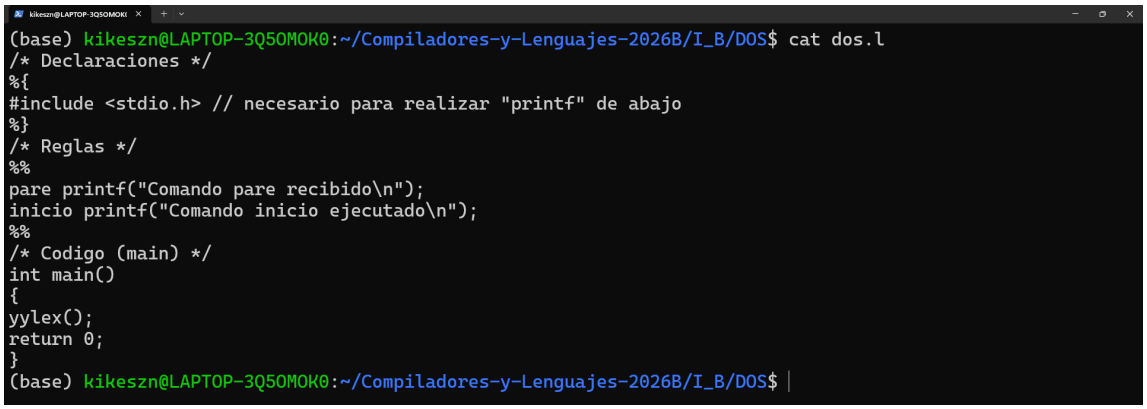


Figura 1: Carpeta y contenido de dos.l

3. Archivo de prueba

Para probar el analizador se crea `entrada.txt` con un párrafo de texto mediante una entrada multilínea en la terminal. Luego se usa `cat entrada.txt` para confirmar el contenido. La captura permite observar las palabras que serán procesadas por las reglas del analizador.

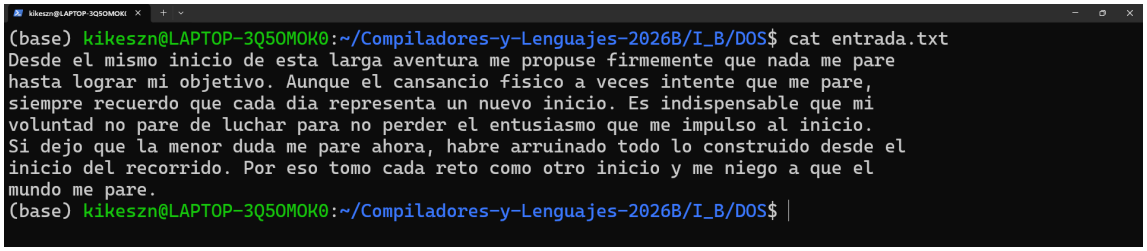


Figura 2: Contenido de entrada.txt

Lex transforma `dos.l` y genera `lex.yy.c`. GCC compila ese código generado y, con `-lfl`, enlaza la biblioteca de Flex necesaria para las funciones del analizador. Si la compilación finaliza sin errores, se obtiene `a.out`.

A continuación, la orden `ls` confirma que `entrada.txt` está en la carpeta de trabajo junto con `a.out` y los archivos generados por Lex, como son `dos.l` y `lex.yy.c`. Este paso prepara una entrada identificable para la redirección de la siguiente ejecución.

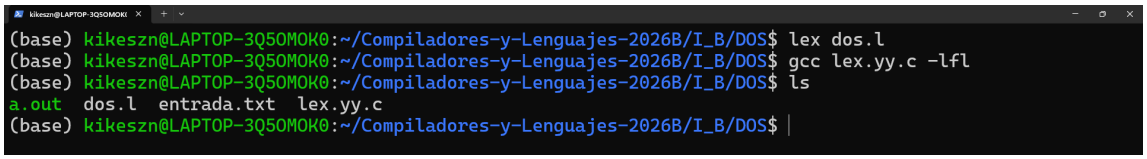


Figura 3: Generación del código C, compilación y archivos resultantes.

4. Ejecución y análisis del resultado

La orden `./a.out <entrada.txt>salida.txt` ejecuta el analizador, toma los datos desde `entrada.txt` y guarda lo que imprime en `salida.txt`. Después, `cat salida.txt` muestra en pantalla el resumen procesado. En la evidencia se reportan 10 reemplazos para las entradas 'pare' e 'inicio' presentes en el párrafo escrito anteriormente.

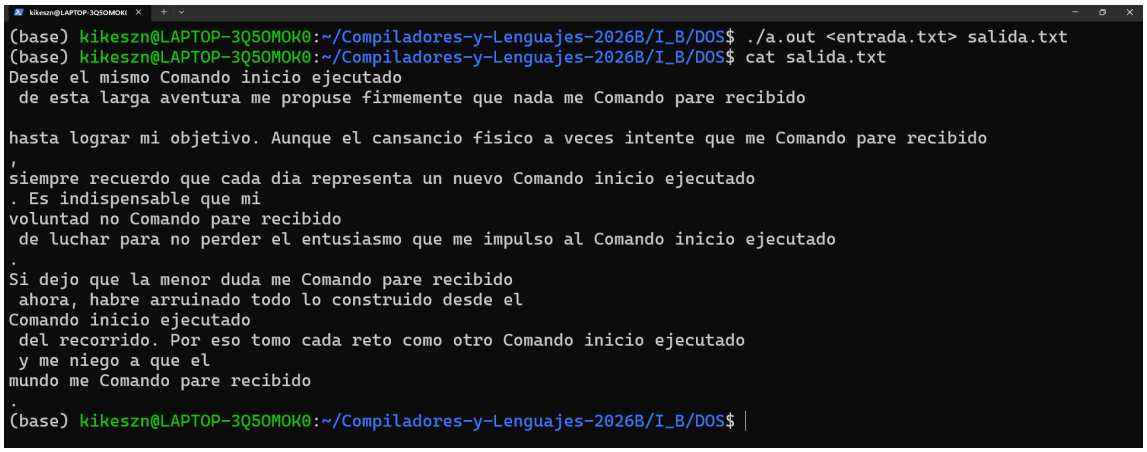


Figura 4: Ejecución con redirección y salida observada.

5. Conclusión

La práctica consolida el flujo para la construcción de un analizador léxico: la definición de patrones, la generación de código en C, su compilación y la validación mediante redirección de archivos. La ejecución nos confirma que para el programa fue fácil reconocer palabras o entradas específicas escritas en un texto, siempre y cuando se describa exactamente la ortografía y sintaxis de lo que se desea reconocer, por ejemplo, de ser el caso, no se reconocería si la palabra inicia con mayúscula o termina con vocal. En conclusión, se demuestra que el analizador generado funciona satisfactoriamente si el objetivo es reconocer cadenas de caracteres o palabras específicas que se definan en el programa fuente.